12

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

직렬화, 파일 업로드, 스트리밍 응답

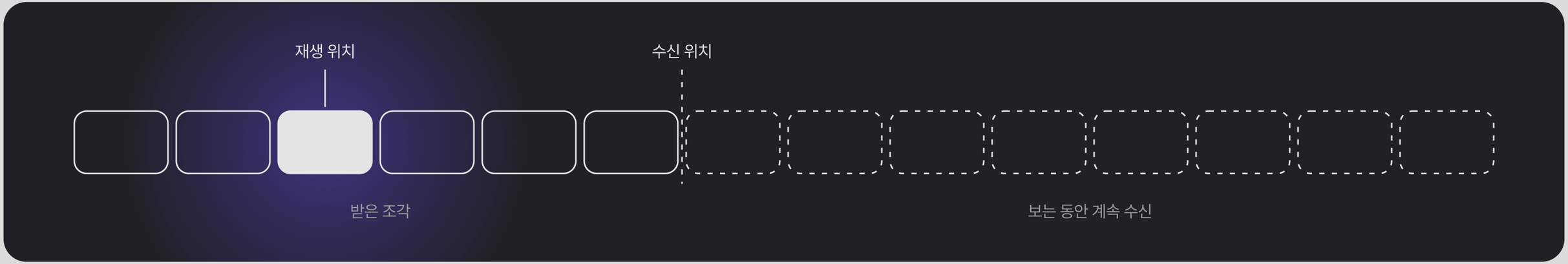
제1부 기반: 백엔드와 Node.js 런타임 · 3장 HTTP 기초

객체와 바이트 사이, 큰 데이터를 주고받는 법



© 한상훈 NestJS 백엔드 마스터

다 받기 전에 재생



→ 2시간짜리 영화: 재생 버튼 → 몇 초 뒤 시작

→ 앞부분 조각부터 받아서 재생, 나머지는 보는 동안 계속 수신

→ 다 받은 뒤 재생이면: 수 GB 다운로드 대기

→ 서버 프로세스가 메모리 부족으로 죽으면: 같은 서버의 결제 요청까지 끊김

© 한상훈 NestJS 백엔드 마스터

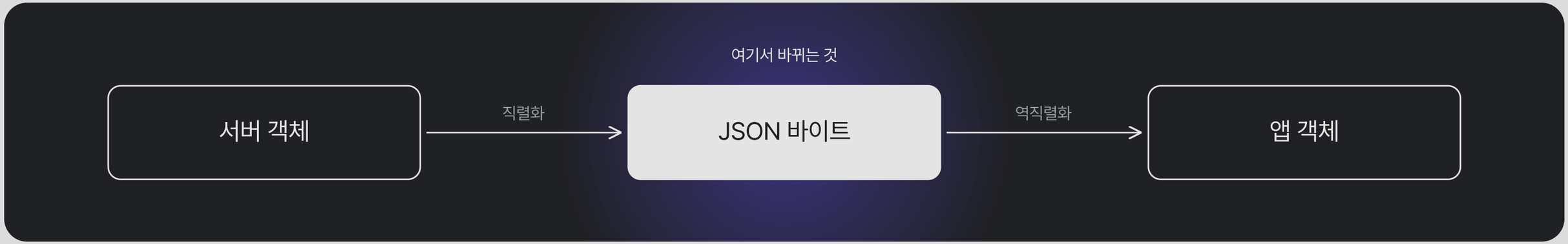
2GB를 메모리에 올리는 서버

동시 접속	통째로 메모리에	조각으로
1명	2GB	조각 몇 개
50명	100GB	조각 몇 개 x 50

→ 계산, 파일 크기 2GB 가정

© 한상훈 NestJS 백엔드 마스터

직렬화



→ 직렬화(serialization): 메모리의 객체 → 보내거나 저장할 바이트

→ 역직렬화: 바이트 → 객체

→ Nest 컨트롤러가 객체를 반환하면: `JSON.stringify`로 직렬화해서 응답

© 한상훈 NestJS 백엔드 마스터

JSON에서 사라지는 것

터미널

```
$ node json-pitfalls.mjs
BigInt: TypeError: Do not know how to serialize a BigInt
큰 정수 ID: 9007199254740992
Date: {"openAt":"2026-11-01T01:00:00.000Z"} string
Map: {"holds":{}}
undefined: {"seat":"A-3-15"}
```

→ 큰 정수 ID: 원래 값 9007199254740993

→ Date: 원래 값 2026-11-01 10:00 (+09:00)

© 한상훈 NestJS 백엔드 마스터

2의 53제곱

`Number.MAX_SAFE_INTEGER`

9007199254740991

→ JavaScript 숫자: 64비트 부동소수점 하나

→ 정확히 셀 수 있는 정수: `Number.MAX_SAFE_INTEGER` = `9007199254740991`까지

→ 그보다 큰 정수: 가까운 다른 정수로 바뀜

→ DB의 64비트 정수 ID(BIGINT)는 이 범위를 넘을 수 있음

→ X(옛 Twitter) API: 숫자 `id`와 함께 문자열 `id_str` 제공 (공식 문서)

© 한상훈 NestJS 백엔드 마스터

ID, 금액, 시각의 표현

값	JSON에서의 문제	보내는 모양
64비트 정수 ID	2^53 넘으면 값이 바뀜	문자열 `"9007199254740993"`
금액	BigInt는 오류, 소수는 오차	원 단위 정수 또는 문자열
시각	Date → UTC 문자열, 받는 쪽은 문자열	오프셋이 붙은 ISO 문자열
Map, Set	빈 객체	배열이나 일반 객체로 변환

→ `BigInt.prototype.toJSON` 덮어쓰기: 프로세스 전체 동작 변경, 응답을 만드는 한 곳에서 변환

© 한상훈 NestJS 백엔드 마스터

좌석 5만 개의 JSON

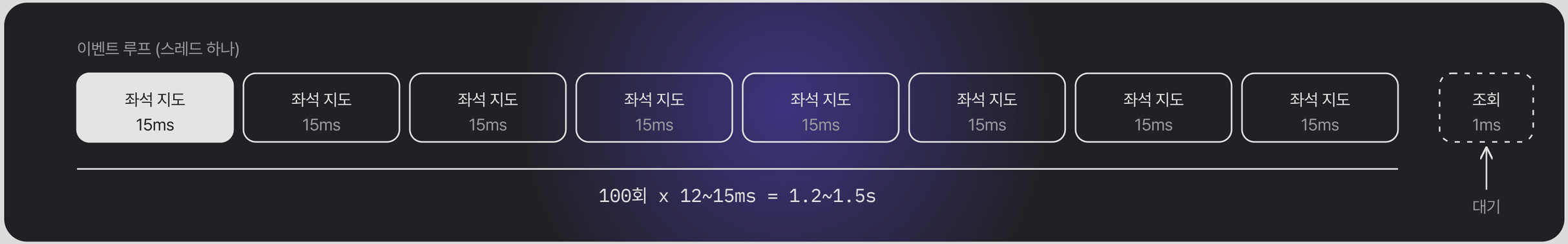
작업	크기	걸린 시간
`JSON.stringify` (객체 → 문자열)	3,115KB	12~15ms
`JSON.parse` (문자열 → 객체)	3,115KB	12~18ms

→ 좌석 지도: 20구역 x 50열 x 50석

데이터 블록: 2026-10-26 강사 PC 실측, 7회 중 가운데 5회 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

그동안 멈추는 요청



→ `JSON.stringify`, `JSON.parse`: 동기 함수, 끝날 때까지 이벤트 루프 점유

→ 좌석 지도 한 번: 이벤트 루프 12~15ms 점유

→ 초당 100번 요청이면: 1.2~1.5초 (계산) → 1초 안에 처리 불가

→ 대응: 결과 문자열이나 압축본을 만들어 두고 재사용, 필요한 구역만 응답

→ 구역 하나만: 2,500석, 크기와 직렬화 시간 약 20분의 1 (계산)

© 한상훈 NestJS 백엔드 마스터

MessagePack

→ 바이너리 직렬화 형식, JSON과 같은 구조(배열, 맵, 문자열, 숫자)

→ 숫자: 텍스트 대신 바이트로, 크기에 맞는 길이

→ 문자열 앞: 길이 표시, 따옴표·쉼표 없음

→ 바이너리 데이터, 64비트 정수, 시각 타입

→ Node 라이브러리: `@msgpack/msgpack`

데이터 블록: @msgpack/msgpack 3.1.3 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

첫 16바이트

```
MessagePack 첫 16바이트: dcc35085a26964a5412d312d31a47a6f
```

→ `dc c350`: 원소 50,000개짜리 배열

→ `85`: 항목 5개짜리 맵

→ `a2 6964`: 2바이트 문자열 `"id"`

→ `a5 412d312d31`: 5바이트 문자열 `"A-1-1"`

→ `a4 7a6f...`: 4바이트 문자열 `"zone"`

→ curl이나 로그로는 읽을 수 없는 바이트

© 한상훈 NestJS 백엔드 마스터

JSON 대 MessagePack

형식	크기	만들기	읽기	brotli 4 뒤
JSON	3,115KB	12~15ms	12~18ms	174KB
MessagePack	2,107KB	12~30ms	17~42ms	190KB

→ 그대로는 MessagePack이 32% 작음

→ 압축 뒤에는 JSON이 더 작음

→ Node에서: JSON 함수가 엔진에 내장, 더 빠름

데이터 블록: 2026-10-26 강사 PC 실측, 같은 좌석 지도 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

MessagePack을 쓰는 곳

상황	맞는 형식
브라우저·앱 공개 API, 압축 있음	JSON
사진, 파일 조각 같은 바이너리 포함	MessagePack (base64 33% 증가 없음)
서버 사이 내부 통신, 압축 없음	MessagePack
캐시 저장 값, 64비트 정수·시각 보존	MessagePack

→ 그대로 왕복: BigInt `{ amount: 10n }`, Date 객체

© 한상훈 NestJS 백엔드 마스터

multipart/form-data

```
POST /concerts/1/poster HTTP/1.1
Content-Type: multipart/form-data; boundary=----X

------X
Content-Disposition: form-data; name="file"; filename="poster.jpg"
Content-Type: image/jpeg

(파일 바이트)
------X--
```

→ 요청 본문 하나에 여러 부분, 경계 문자열(boundary)로 구분

→ 부분마다 이름, 파일 이름, 타입, 내용

© 한상훈 NestJS 백엔드 마스터

FileInterceptor

```
@Post('poster')
@UseInterceptors(FileInterceptor('file', { limits: { fileSize: 5 * MB } }))
poster(@UploadedFile() file: Express.Multer.File) {
  return { name: file.originalname, size: file.size, mimetype: file.mimetype };
}
```

○ ○ ○ 터미널

```
$ curl -F file=@poster-3mb.jpg localhost:4012/concerts/1/poster
{"name":"poster-3mb.jpg","size":3145728,"mimetype":"image/jpeg"}
```

→ multer: Express용 multipart 처리 라이브러리, `@nestjs/platform-express`에 포함

→ 저장 위치 기본값: 메모리(`file.buffer`)

© 한상훈 NestJS 백엔드 마스터

다 받은 뒤의 검사

```
@Post('poster-unlimited')
@UseInterceptors(FileInterceptor('file'))
posterUnlimited(
  @UploadedFile(new ParseFilePipe({ validators: [new MaxFileSizeValidator({ maxSize: 5 * MB })] }))
  file: Express.Multer.File,
) { ... }
```

○ ○ ○ 터미널

```
$ curl -F file=@poster-200mb.jpg localhost:4012/concerts/1/poster-unlimited
{"type":"about:blank","title":"Bad Request","status":400,
 "detail":"Validation failed (current file size is 209715200, expected size is less than 5242880)", ...}
```

→ 서버 메모리: 시작 95MB → 최고 500MB, 응답 뒤에도 499MB

→ 검사 파이프: 파일을 다 받은 뒤에 실행

© 한상훈 NestJS 백엔드 마스터

받는 도중의 상한

```
@UseInterceptors(FileInterceptor('file', { limits: { fileSize: 5 * MB } }))
```

터미널

```
$ curl -F file=@poster-200mb.jpg localhost:4012/concerts/1/poster
{"type":"about:blank","title":"Payload Too Large","status":413,
 "detail":"File too large","instance":"/concerts/1/poster"}
```

→ 서버 메모리: 최고 154~177MB, 응답 뒤 98~112MB

→ 5MB를 넘는 순간 받기 중단, 413

© 한상훈 NestJS 백엔드 마스터

업로드 메모리 비교

200MB 업로드	응답	서버 메모리 최고	응답 뒤
검사 파이프만	400	499~500MB	499MB
multer `limits`	413	154~177MB	98~112MB

→ 시작 메모리 약 95MB

데이터 블록: 2026-10-26 강사 PC 실측, process.memoryUsage().rss를 50ms마다 기록 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

확장자로 정해진 image/jpeg

→ 시연 파일: 무작위 바이트, 이름만 .jpg

→ 서버가 받은 mimetype: `image/jpeg`

→ mimetype: 클라이언트가 보낸 값, curl은 확장자로 결정

→ 파일 종류 확인: 파일 앞부분 바이트(매직 넘버) 직접 검사

→ 큰 파일: 디스크 저장, 또는 서버를 거치지 않는 객체 스토리지 직접 업로드

© 한상훈 NestJS 백엔드 마스터

통째로 보내기

```
@Get('sales-export-buffer')
async exportBuffer(@Res() res: Response) {
  const data = await readFile(EXPORT_FILE);
  res.type('text/csv').send(data);
}
```

→ `readFile`: 파일 전체를 메모리에 읽은 뒤 반환

© 한상훈 NestJS 백엔드 마스터

흘려보내기

```
@Get('sales-export')
exportStream(@Param('id') id: string) {
  return new StreamableFile(createReadStream(EXPORT_FILE), {
    type: 'text/csv',
    disposition: `attachment; filename="sales-${id}.csv"`,
  });
}
```

→ `createReadStream`: 파일을 조각(기본 64KB)으로 읽는 스트림

→ `StreamableFile`: Nest가 스트림을 응답으로 흘려보내게 하는 래퍼

→ disposition attachment: 브라우저가 `sales-1.csv` 파일로 저장

© 한상훈 NestJS 백엔드 마스터

200MB 판매 내역

내려받기 212,136,890 bytes	통째로 (readFile)	흘려보내기 (stream)
한 명, 메모리 최고	299MB	96~147MB
다섯 명 동시, 메모리 최고	1,108~1,109MB	169~172MB
첫 바이트까지	0.17~0.32초	0.007~0.022초

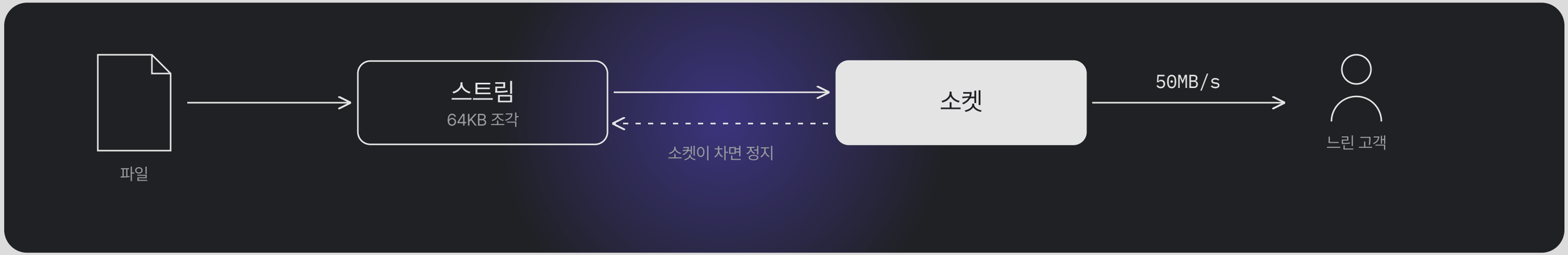
→ 시작 메모리 약 95MB

→ 판매 내역: 320만 줄짜리 CSV (시연용 생성)

데이터 블록: 2026-10-26 강사 PC 실측 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

느린 고객



→ 초당 50MB로 받는 고객 (`curl --limit-rate 50M`): 4.06초, 메모리 최고 163MB

→ 스트림: 고객이 못 받으면 파일 읽기 일시 정지, 받으면 재개

→ 통째로: 고객이 다 받을 때까지 파일 전체가 메모리에

→ 스트림이 필요한 응답: 크기가 정해져 있지 않은 것 (판매 내역, 원본 파일, 기간 내보내기)

→ 몇 KB짜리 응답: 객체 반환 그대로

© 한상훈 NestJS 백엔드 마스터

세 경계

경계	정할 것	이 프로젝트의 결정
객체 → 바이트	타입 표현	ID 문자열, 금액 원 단위 정수, 시각 오프셋 ISO
받기	크기 상한 위치	multer `limits` 5MB, 넘으면 413
보내기	메모리에 올리는 양	큰 파일은 `StreamableFile`

코드 리뷰에서 볼 것	물어볼 것
컨트롤러 근처의 `readFile`	파일이 얼마나 커질 수 있나
요청마다 큰 배열 `JSON.stringify`	미리 만들어 둘 수 있나
`limits` 없는 `FileInterceptor`	상한 누락
메모리 그래프의 계단	통째로 올리는 곳

→ 큰 JSON: 만들어 둔 문자열·압축본 재사용

© 한상훈 NestJS 백엔드 마스터

조각으로 흘려보내기

"큰 데이터는 메모리에 다 올리지 않고,
받는 쪽 속도에 맞춰 조각으로 흘려보냅니다."